# PROBLEM-SHEET-1

**Practical 1:**

**(a) Install MongoDB Community Edition and PHP MongoDB Driver.**
**(b) Create a database named 'pg_nosql'.**
**(c) Create a collection 'student' with fields: enrollment no, name, email, course, and year.**
**(d) Insert at least five student documents.**
**(e) Display all inserted documents using MongoDB shell and PHP.**

## 1. Install MongoDB Community Editon

**Step 1: Go to MongoDB's official site:**

https://www.mongodb.com/try/download/community

**Step 2: Download and run the Installer**

**Step 3: Verify the Installation:**

**Open command prompt and enter:**

mongod --version

**Expected output:**

```
C:\Users\KARAN>mongod --version
db version v8.0.11
Build Info: {
    "version": "8.0.11",
    "gitVersion": "bed99f699da6cb2b74262aa6d473446c41476643",
    "modules": [],
    "allocator": "tcmalloc-gperf",
    "environment": {
        "distmod": "windows",
        "distarch": "x86_64",
        "target_arch": "x86_64"
    }
}
```

## 2. Install PHP MongoDB Driver

**Step 1: Go to following site and download DLL (according to your PHP version)**

https://pecl.php.net/package/mongodb

**Download the file that matches:**

- PHP version
- Thread Safe (TS) or Non-Thread Safe (NTS)
- x64

**Step 2: Extract DLL (php_mongodb.dll)**
**Paste into: C:\php\ext\ (or your installed path)**

**Step 3: Enable extension:**

open **php.ini** and add: **extension=mongodb**

**Step 4: Verify Extension**

Open command prompt and enter: extension=mongodb

It must display **MongoDB**

**Step 5: Install MongoDB PHP Library (Composer)**

**Add following command: (make sure composer is installed)**

composer require mongodb/mongodb

## (b) Create a database named 'pg_nosql'

**Open mongosh (from compass or cmd) and enter following command**

```
test> use pg_nosql
switched to db pg_nosql
```

## (c) Create Collection student

```
pg_nosql> db.createCollection("student")
{ ok: 1 }
```

## (d) Insert Five Documents

```
pg_nosql> db.student.insertMany([
...   {
...     enrollment_no: 101,
...     name: "Karan",
...     email: "karan@gmail.com",
...     course: "BCA",
...     year: 2
...   },
...   {
...     enrollment_no: 102,
...     name: "Amit",
...     email: "amit@gmail.com",
...     course: "BCA",
...     year: 3
...   },
...   {
...     enrollment_no: 103,
...     name: "Riya",
...     email: "riya@gmail.com",
...     course: "BSc IT",
...     year: 1
...   },
...   {
...     enrollment_no: 104,
...     name: "Neha",
...     email: "neha@gmail.com",
...     course: "BSc IT",
...     year: 2
...   },
...   {
...     enrollment_no: 105,
...     name: "Rahul",
...     email: "rahul@gmail.com",
...     course: "BCA",
...     year: 1
...   }
...   ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('699ebcf95301bfd027718dc4'),
    '1': ObjectId('699ebcf95301bfd027718dc5'),
    '2': ObjectId('699ebcf95301bfd027718dc6'),
    '3': ObjectId('699ebcf95301bfd027718dc7'),
    '4': ObjectId('699ebcf95301bfd027718dc8')
  }
}
```

## (e) Display Documents Using Mongo Shell

```
pg_nosql> db.student.find().pretty()
[
  {
    _id: ObjectId('699ebcf95301bfd027718dc4'),
    enrollment_no: 101,
    name: 'Karan',
    email: 'karan@gmail.com',
    course: 'BCA',
    year: 2
  },
  {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit@gmail.com',
    course: 'BCA',
    year: 3
  },
  {
    _id: ObjectId('699ebcf95301bfd027718dc6'),
    enrollment_no: 103,
    name: 'Riya',
    email: 'riya@gmail.com',
    course: 'BSc IT',
    year: 1
  },
  {
    _id: ObjectId('699ebcf95301bfd027718dc7'),
    enrollment_no: 104,
    name: 'Neha',
    email: 'neha@gmail.com',
    course: 'BSc IT',
    year: 2
  },
  {
    _id: ObjectId('699ebcf95301bfd027718dc8'),
    enrollment_no: 105,
    name: 'Rahul',
    email: 'rahul@gmail.com',
    course: 'BCA',
    year: 1
  }
]
pg_nosql> |
```

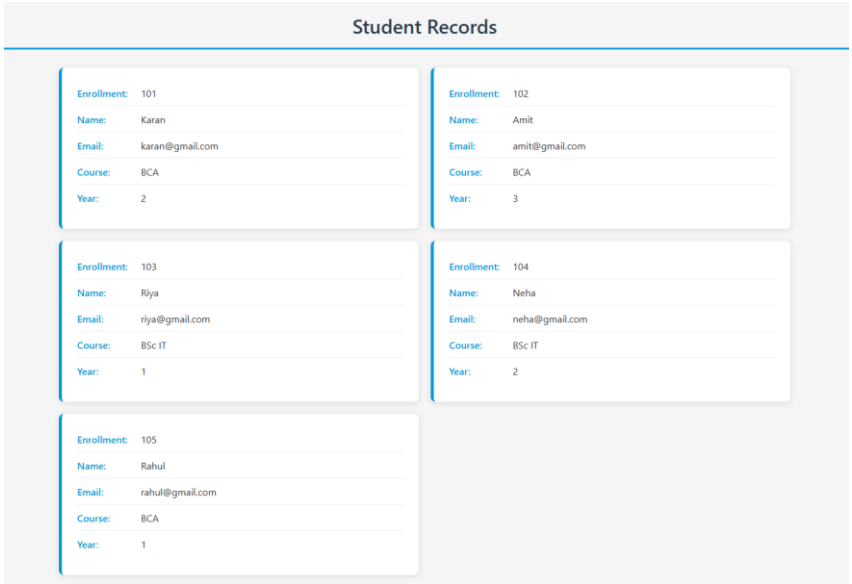
## Display Using PHP

**display.php**

```php
<?php
require '../vendor/autoload.php';

$client = new MongoDB\Client("mongodb://localhost:27017");
$collection = $client->pg_nosql->student;

$students = $collection->find();

echo "<!DOCTYPE html>";
echo "<html>";
echo "<head>";
echo "<link rel='stylesheet' href='style.css'>";
echo "</head>";
echo "<body>";

echo "<h2>Student Records</h2>";
echo "<div class='student-container'>";

foreach ($students as $student) {
   echo "<div class='student-card'>";
   echo "<p><strong>Enrollment:</strong> " . $student['enrollment_no'] . "</p>";
   echo "<p><strong>Name:</strong> " . $student['name'] . "</p>";
```

```
    echo "<p><strong>Email:</strong> " . $student['email'] . "</p>";
    echo "<p><strong>Course:</strong> " . $student['course'] . "</p>";
    echo "<p><strong>Year:</strong> " . $student['year'] . "</p>";
    echo "</div>";
}

echo "</div>";
echo "</body>";
echo "</html>";
?>
```

**OUTPUT:**



**Practical 2:**

**(a) Insert a new student record into the 'students' collection.**

**(b) Retrieve and display all student records in tabular format.**

**(c) Update the email ID of a student using enrollment number.**

**(d) Delete a student record based on enrollment number.**

### (a) Insert New Student Record (MongoDB Shell)

```
> db.student.insertOne({
    enrollment_no: 106,
    name: "Priya",
    email: "priya@gmail.com",
    course: "BCA",
    year: 2
  })
< {
    acknowledged: true,
    insertedId: ObjectId('699ec0c5c93e1aaa9f3ce781')
  }
pg_nosql>
```

### (b) Retrieve & Display All Records in Tabular Format

**display.php:**

```
<?php
require '../vendor/autoload.php';

$client = new MongoDB\Client("mongodb://localhost:27017");
$collection = $client->pg_nosql->student;

$students = $collection->find();
?>
<!DOCTYPE html>
<html>
<head>
  <link rel='stylesheet' href='style.css'>
</head>
<body>
  <h2>Student Table</h2>
  <table>
    <tr>
      <th>Enrollment No</th>
      <th>Name</th>
      <th>Email</th>
      <th>Course</th>
      <th>Year</th>
    </tr>
    <?php foreach ($students as $s): ?>
```

```
        <tr>
          <td><?= $s['enrollment_no'] ?></td>
          <td><?= $s['name'] ?></td>
          <td><?= $s['email'] ?></td>
          <td><?= $s['course'] ?></td>
          <td><?= $s['year'] ?></td>
        </tr>
        <?php endforeach; ?>
      </table>
    </body>
    </html>
```

**OUTPUT:**

Student Table

| ENROLLMENT NO | NAME | EMAIL | COURSE | YEAR |
|---|---|---|---|---|
| 101 | Karan | karan@gmail.com | BCA | 2 |
| 102 | Amit | amit@gmail.com | BCA | 3 |
| 103 | Riya | riya@gmail.com | BSc IT | 1 |
| 104 | Neha | neha@gmail.com | BSc IT | 2 |
| 105 | Rahul | rahul@gmail.com | BCA | 1 |
| 106 | Priya | priya@gmail.com | BCA | 2 |

### (c) Update Email Using Enrollment Number

```
> db.student.updateOne(
    { enrollment_no: 102 },
    { $set: { email: "amit123@gmail.com" } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.student.find({ enrollment_no: 102 })
< {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit123@gmail.com',
    course: 'BCA',
    year: 3
  }
```

**(d) Delete Student Using Enrollment Number**

```
> db.student.deleteOne({ enrollment_no: 105 })
< {
    acknowledged: true,
    deletedCount: 1
  }
> db.student.find({ enrollment_no: 105 })
<
pg_nosql> /
```

**Practical 3:**
**(a) Create an index on the enrollment_no field of the 'students' collection.**
**(b) Execute a search query before and after indexing.**
**(c) Use explain() to compare query execution plans.**

**(a) Create Index on enrollment_no**

```
> db.student.createIndex({ enrollment_no: 1 })
< enrollment_no_1
pg_nosql>
```

**(b) Execute Search Query BEFORE & AFTER Index**

first drop index to simulate before indexing:

```
> db.student.dropIndex("enrollment_no_1")
< { nIndexesWas: 2, ok: 1 }
pg_nosql>
```

Now run search:

```
> db.student.find({ enrollment_no: 102 })
< {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit123@gmail.com',
    course: 'BCA',
    year: 3
  }
```

Recreate index:

```
> db.student.createIndex({ enrollment_no: 1 })
< enrollment_no_1
pg_nosql>
```

Run same search again:

```
> db.student.find({ enrollment_no: 102 })
< {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit123@gmail.com',
    course: 'BCA',
    year: 3
  }
pg_nosql>
```

(Visually same result — difference is in **performance**, not output)

**(c) Use explain() to Compare Execution Plans**

**Before index**

```
"stage" : "COLLSCAN"
```

Means → Collection Scan (slow)

**After index**

```
stage: 'IXSCAN',
```

Means → Index Scan (fast)

**Practical 4:**

**(a) Design MongoDB collections for an Online Course Management System: users, courses, enrollments.**

**(b) Insert sample documents into each collection.**

**(c) Retrieve all courses enrolled by a specific user.**

**(d) Update course progress for a user.**

## (a) Design Collections

MongoDB creates collection automatically when data is inserted.

## (b) Insert Sample Documents

```
> db.users.insertMany([
    { user_id: 1, name: "Karan", email: "karan@gmail.com" },
    { user_id: 2, name: "Amit", email: "amit@gmail.com" }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('699ec37fa6d3ebb9f164fe2a'),
      '1': ObjectId('699ec37fa6d3ebb9f164fe2b')
    }
  }
```

```
> db.courses.insertMany([
    { course_id: 101, title: "NodeJS", duration: "3 Months" },
    { course_id: 102, title: "MongoDB", duration: "2 Months" }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('699ec3aaa6d3ebb9f164fe2c'),
      '1': ObjectId('699ec3aaa6d3ebb9f164fe2d')
    }
  }
```

```
> db.enrollments.insertMany([
    { user_id: 1, course_id: 101, progress: 40 },
    { user_id: 1, course_id: 102, progress: 20 },
    { user_id: 2, course_id: 101, progress: 60 }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('699ec3bba6d3ebb9f164fe2e'),
      '1': ObjectId('699ec3bba6d3ebb9f164fe2f'),
      '2': ObjectId('699ec3bba6d3ebb9f164fe30')
    }
  }
```

## (c) Retrieve All Courses Enrolled by Specific User

```
> db.enrollments.aggregate([
    { $match: { user_id: 1 } },
    {
      $lookup: {
        from: "courses",
        localField: "course_id",
        foreignField: "course_id",
        as: "course_details"
      }
    }
  ])
```

```
{
  _id: ObjectId('699ec3bba6d3ebb9f164fe2e'),
  user_id: 1,
  course_id: 101,
  progress: 40,
  course_details: [
    {
      _id: ObjectId('699ec3aaa6d3ebb9f164fe2c'),
      course_id: 101,
      title: 'NodeJS',
      duration: '3 Months'
    }
  ]
}
{
  _id: ObjectId('699ec3bba6d3ebb9f164fe2f'),
  user_id: 1,
  course_id: 102,
  progress: 20,
  course_details: [
    {
      _id: ObjectId('699ec3aaa6d3ebb9f164fe2d'),
      course_id: 102,
      title: 'MongoDB',
      duration: '2 Months'
    }
  ]
}
```

## (d) Update Course Progress for a User

```
> db.enrollments.updateOne(
    { user_id: 1, course_id: 101 },
    { $set: { progress: 80 } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

```
> db.enrollments.find({ user_id: 1 })
< {
    _id: ObjectId('699ec3bba6d3ebb9f164fe2e'),
    user_id: 1,
    course_id: 101,
    progress: 80
  }
  {
    _id: ObjectId('699ec3bba6d3ebb9f164fe2f'),
    user_id: 1,
    course_id: 102,
    progress: 20
  }
```

**Practical 5:**
**(a) Embed address details inside a user document.**
**(b) Reference course IDs inside the enrollments collection.**
**(c) Demonstrate retrieval of embedded and referenced data using PHP.**

### (a) Embed Address Inside User Document

```
> db.users.insertOne({
    user_id: 3,
    name: "Karan",
    email: "karan@gmail.com",
    address: {
      city: "Surat",
      state: "Gujarat",
      pincode: 395006
    }
  })
< {
    acknowledged: true,
    insertedId: ObjectId('699ec5208037dc979c8add3e')
  }
pg_nosql>
```

### (b) Reference Course IDs Inside Enrollments

```
> db.enrollments.insertOne({
    user_id: 3,
    course_id: 101,
    progress: 30
  })
< {
    acknowledged: true,
    insertedId: ObjectId('699ec5428037dc979c8add3f')
  }
pg_nosql>
```

### (c) Retrieve Embedded & Referenced Data Using PHP

**retrieve.php**

```php
<?php
require '../vendor/autoload.php';

$client = new MongoDB\Client("mongodb://localhost:27017");
$db = $client->pg_nosql;

// Retrieve Embedded Data (User + Address)
$user = $db->users->findOne(['user_id' => 3]);

// Retrieve Referenced Data (Enrollment + Course)
$enrollment = $db->enrollments->findOne(['user_id' => 3]);
```

```
$course = $db->courses->findOne(['course_id' => $enrollment['course_id']]);
?>
<!DOCTYPE html>
<html>
<head>
  <link rel='stylesheet' href='style.css'>
</head>
<body>
  <h2>User Data</h2>

  <div class="data-section">
    <h3>Embedded Data</h3>
    <div class="data-card">
      <p><strong>Name:</strong> <?= $user['name'] ?></p>
      <p><strong>City:</strong> <?= $user['address']['city'] ?></p>
      <p><strong>State:</strong> <?= $user['address']['state'] ?></p>
    </div>
  </div>

  <div class="data-section">
    <h3>Referenced Data</h3>
    <div class="data-card">
      <p><strong>Course Title:</strong> <?= $course['title'] ?></p>
      <p><strong>Progress:</strong> <?= $enrollment['progress'] ?>%</p>
    </div>
  </div>
</body>
</html>
```

**OUTPUT:**

**Practical 6:**

**(a) Retrieve students whose year is greater than 1 using $gt.**

**(b) Fetch students enrolled in specific courses using $in.**

**(c) Sort students by name and year.**

**(d) Implement pagination using skip() and limit().**

**(a) Students whose year > 1 using $gt**

**Enter following command**

```
> db.student.find({ year: { $gt: 1 } })
```

**Output**

```
< {
    _id: ObjectId('699ebcf95301bfd027718dc4'),
    enrollment_no: 101,
    name: 'Karan',
    email: 'karan@gmail.com',
    course: 'BCA',
    year: 2
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit123@gmail.com',
    course: 'BCA',
    year: 3
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc7'),
    enrollment_no: 104,
    name: 'Neha',
    email: 'neha@gmail.com',
    course: 'BSc IT',
    year: 2
  }
  {
    _id: ObjectId('699ec0c5c93e1aaa9f3ce781'),
    enrollment_no: 106,
    name: 'Priya',
    email: 'priya@gmail.com',
    course: 'BCA',
    year: 2
  }
```

## (b) Students enrolled in specific courses using $in

```
> db.student.find({
    course: { $in: ["BCA"] }
  })
< {
    _id: ObjectId('699ebcf95301bfd027718dc4'),
    enrollment_no: 101,
    name: 'Karan',
    email: 'karan@gmail.com',
    course: 'BCA',
    year: 2
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit123@gmail.com',
    course: 'BCA',
    year: 3
  }
  {
    _id: ObjectId('699ec0c5c93e1aaa9f3ce781'),
    enrollment_no: 106,
    name: 'Priya',
    email: 'priya@gmail.com',
    course: 'BCA',
    year: 2
  }
```

## (c) Sort students by name and year

```
> db.student.find().sort({ name: 1, year: 1 })
```

```
< {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit123@gmail.com',
    course: 'BCA',
    year: 3
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc4'),
    enrollment_no: 101,
    name: 'Karan',
    email: 'karan@gmail.com',
    course: 'BCA',
    year: 2
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc7'),
    enrollment_no: 104,
    name: 'Neha',
    email: 'neha@gmail.com',
    course: 'BSc IT',
    year: 2
  }
```

```
  {
    _id: ObjectId('699ec0c5c93e1aaa9f3ce781'),
    enrollment_no: 106,
    name: 'Priya',
    email: 'priya@gmail.com',
    course: 'BCA',
    year: 2
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc6'),
    enrollment_no: 103,
    name: 'Riya',
    email: 'riya@gmail.com',
    course: 'BSc IT',
    year: 1
  }
```

### (d) Pagination using skip() and limit()

```
> db.student.find().skip(0).limit(2)
< {
    _id: ObjectId('699ebcf95301bfd027718dc4'),
    enrollment_no: 101,
    name: 'Karan',
    email: 'karan@gmail.com',
    course: 'BCA',
    year: 2
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc5'),
    enrollment_no: 102,
    name: 'Amit',
    email: 'amit123@gmail.com',
    course: 'BCA',
    year: 3
  }
> db.student.find().skip(2).limit(2)
< {
    _id: ObjectId('699ebcf95301bfd027718dc6'),
    enrollment_no: 103,
    name: 'Riya',
    email: 'riya@gmail.com',
    course: 'BSc IT',
    year: 1
  }
  {
    _id: ObjectId('699ebcf95301bfd027718dc7'),
    enrollment_no: 104,
    name: 'Neha',
    email: 'neha@gmail.com',
    course: 'BSc IT',
    year: 2
```

**Practical 7:**

**(a) Create an 'orders' collection with fields: order_id, customer_id, product, quantity, price, order_date.**

**(b) Insert sample order data.**

**(c) Retrieve all orders of a specific customer.**

**(d) Retrieve orders placed within a given date range.**

### (a) Design Collections

MongoDB creates collection automatically when data is inserted.

### (b) Insert Sample Orders

```
> db.orders.insertMany([
    {
      order_id: 1,
      customer_id: 101,
      product: "Laptop",
      quantity: 1,
      price: 50000,
      order_date: ISODate("2025-02-10")
    },
    {
      order_id: 2,
      customer_id: 102,
      product: "Mobile",
      quantity: 2,
      price: 20000,
      order_date: ISODate("2025-02-15")
    },
    {
      order_id: 3,
      customer_id: 101,
      product: "Tablet",
      quantity: 1,
      price: 15000,
      order_date: ISODate("2025-03-01")
    },
    {
      order_id: 4,
      customer_id: 103,
      product: "Headphones",
      quantity: 3,
      price: 3000,
      order_date: ISODate("2025-03-05")
    }
```

### (c) Retrieve Orders of Specific Customer

```
> db.orders.find({ customer_id: 101 })
< {
    _id: ObjectId('699ecd411206048fd4f0a8b8'),
    order_id: 1,
    customer_id: 101,
    product: 'Laptop',
    quantity: 1,
    price: 50000,
    order_date: 2025-02-10T00:00:00.000Z
  }
  {
    _id: ObjectId('699ecd411206048fd4f0a8ba'),
    order_id: 3,
    customer_id: 101,
    product: 'Tablet',
    quantity: 1,
    price: 15000,
    order_date: 2025-03-01T00:00:00.000Z
  }
```

### (d) Retrieve Orders Within Date Range

```
> db.orders.find({
    order_date: {
      $gte: ISODate("2025-02-01"),
      $lte: ISODate("2025-02-28")
    }
  })
< {
    _id: ObjectId('699ecd411206048fd4f0a8b8'),
    order_id: 1,
    customer_id: 101,
    product: 'Laptop',
    quantity: 1,
    price: 50000,
    order_date: 2025-02-10T00:00:00.000Z
  }
  {
    _id: ObjectId('699ecd411206048fd4f0a8b9'),
    order_id: 2,
    customer_id: 102,
    product: 'Mobile',
    quantity: 2,
    price: 20000,
    order_date: 2025-02-15T00:00:00.000Z
  }
```

**Practical 8:**

**(a) Calculate total sales per product using aggregation pipeline.**

**(b) Find average order value per customer.**

**(c) Identify top three customers based on total purchase amount.**

## (a) Calculate Total Sales Per Product

```
> db.orders.aggregate([
    {
      $group: {
        _id: "$product",
        totalSales: {
          $sum: { $multiply: ["$quantity", "$price"] }
        }
      }
    }
  ])
< {
    _id: 'Headphones',
    totalSales: 9000
  }
  {
    _id: 'Laptop',
    totalSales: 50000
  }
  {
    _id: 'Mobile',
    totalSales: 40000
  }
  {
    _id: 'Tablet',
    totalSales: 15000
  }
```

## (b) Find Average Order Value Per Customer

```
> db.orders.aggregate([
    {
      $group: {
        _id: "$customer_id",
        avgOrderValue: {
          $avg: { $multiply: ["$quantity", "$price"] }
        }
      }
    }
  ])
< {
    _id: 101,
    avgOrderValue: 32500
  }
  {
    _id: 102,
    avgOrderValue: 40000
  }
  {
    _id: 103,
    avgOrderValue: 9000
  }
```

### (c) Top 3 Customers by Total Purchase Amount

```
> db.orders.aggregate([
    {
      $group: {
        _id: "$customer_id",
        totalPurchase: {
          $sum: { $multiply: ["$quantity", "$price"] }
        }
      }
    },
    { $sort: { totalPurchase: -1 } },
    { $limit: 3 }
  ])
< {
    _id: 101,
    totalPurchase: 65000
  }
  {
    _id: 102,
    totalPurchase: 40000
  }
  {
    _id: 103,
    totalPurchase: 9000
  }
```

**Practical 9:**

**(a) Generate website log data with fields: IP address, page URL, timestamp.**

**(b) Store logs in MongoDB.**

**(c) Count page visits per URL using aggregation framework.**

### (a) Generate Website Log Data

**Each log document will look like:**

```
{
 ip: "192.168.1.1",
 url: "/home",
 timestamp: ISODate("2025-03-01T10:00:00Z")
}
```

### (b) Store Logs in MongoDB

```
> db.weblogs.insertMany([
    { ip: "192.168.1.1", url: "/home", timestamp: new Date() },
    { ip: "192.168.1.2", url: "/about", timestamp: new Date() },
    { ip: "192.168.1.3", url: "/home", timestamp: new Date() },
    { ip: "192.168.1.4", url: "/contact", timestamp: new Date() },
    { ip: "192.168.1.5", url: "/home", timestamp: new Date() },
    { ip: "192.168.1.6", url: "/about", timestamp: new Date() }
  ])
```

### (c) Count Page Visits Per URL (Aggregation)

```
> db.weblogs.aggregate([
    {
      $group: {
        _id: "$url",
        visitCount: { $sum: 1 }
      }
    }
  ])
< {
    _id: '/about',
    visitCount: 2
  }
  {
    _id: '/contact',
    visitCount: 1
  }
  {
    _id: '/home',
    visitCount: 3
  }
```

**Practical 10:**

**(a) Write MapReduce functions to calculate total visits per web page.**

**(b) Execute MapReduce using MongoDB and PHP.**

**(c) Display the output collection.**

### (a) Write MapReduce Functions

```
> var mapFunction = function() {
    emit(this.url, 1);
  };
```

```
> var reduceFunction = function(key, values) {
    return Array.sum(values);
  };
```

### (b) Execute MapReduce (MongoDB Shell)

```
> db.weblogs.mapReduce(
    mapFunction,
    reduceFunction,
    {
      out: "page_visit_counts"
    }
  );
< DeprecationWarning: Collection.mapReduce() is deprecated. Use an aggregation instead.
  See https://mongodb.com/docs/manual/core/map-reduce for details.
< { result: 'page_visit_counts', ok: 1 }
```

### (c) Display Output Collection (mongosh)

```
> db.page_visit_counts.find()
< {
    _id: '/about',
    value: 2
  }
  {
    _id: '/home',
    value: 3
  }
  {
    _id: '/contact',
    value: 1
  }
```

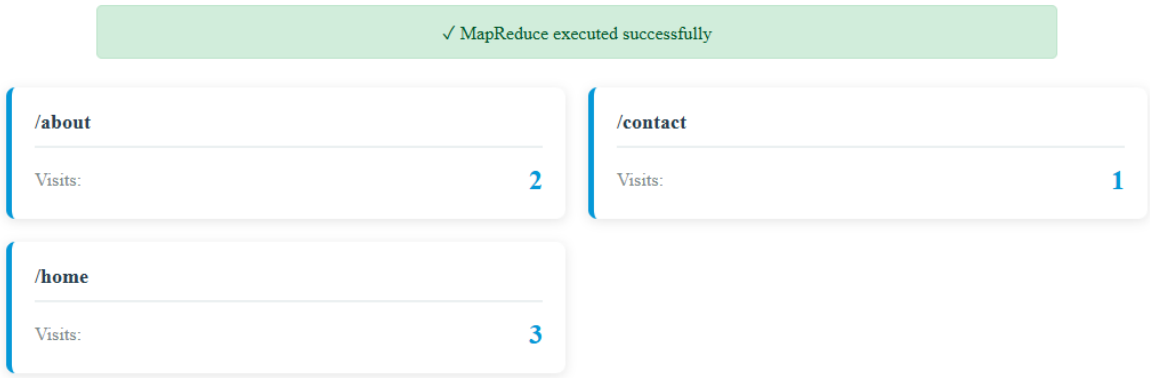
### Execute MapReduce Using PHP

**mapreduce.php**

```
<?php
require '../vendor/autoload.php';

$client = new MongoDB\Client("mongodb://localhost:27017");
$db = $client->pg_nosql;

// Map function
$map = new MongoDB\BSON\Javascript("
  function() {
    emit(this.url, 1);
  }
");

// Reduce function
$reduce = new MongoDB\BSON\Javascript("
  function(key, values) {
    return Array.sum(values);
  }
");

// Execute MapReduce
$result = $db->command([
  'mapReduce' => 'weblogs',
  'map' => $map,
  'reduce' => $reduce,
  'out' => 'page_visit_counts_php'
]);
```

```
// Display result
$collection = $db->page_visit_counts_php;
$docs = $collection->find();
?>
<!DOCTYPE html>
<html>
<head>
  <link rel='stylesheet' href='style.css'>
</head>
<body>
  <h2>Page Visit Statistics</h2>
  <p class="success-message">✓ MapReduce executed successfully</p>

  <div class="stats-container">
    <?php foreach ($docs as $doc): ?>
    <div class="stat-card">
      <div class="stat-url"><?= $doc['_id'] ?></div>
      <div class="stat-visits">
        <span class="visits-label">Visits:</span>
        <span class="visits-number"><?= $doc['value'] ?></span>
      </div>
    </div>
    <?php endforeach; ?>
  </div>
</body>
</html>
```

**OUTPUT:**



**Practical 11:**

**(a) Create a 'products' collection with fields: product_id, name, category, price, stock, and supplier details (embedded document with supplier_name, contact, address).**

**(b) Insert at least 7 product documents across different categories (Electronics, Books, Clothing).**

**(c) Retrieve products where stock is less than 10 using comparison operators.**

**(d) Update the price of all products in a specific category by increasing it by 10%.**

## (a) Create products Collection Structure

MongoDB creates collection automatically on insert.

## (b) Insert 7 Products

```
> db.products.insertMany([
  {
    product_id: 1,
    name: "Laptop",
    category: "Electronics",
    price: 50000,
    stock: 5,
    supplier: { supplier_name: "TechSupplier", contact: "9876543210", address: "Mumbai" }
  },
  {
    product_id: 2,
    name: "Mobile",
    category: "Electronics",
    price: 20000,
    stock: 15,
    supplier: { supplier_name: "GadgetWorld", contact: "9123456789", address: "Delhi" }
  },
  {
    product_id: 3,
    name: "Headphones",
    category: "Electronics",
    price: 3000,
    stock: 8,
    supplier: { supplier_name: "SoundHub", contact: "9988776655", address: "Pune" }
  },
```

```
  {
    product_id: 4,
    name: "Novel Book",
    category: "Books",
    price: 500,
    stock: 20,
    supplier: { supplier_name: "BookStore", contact: "9001122334", address: "Ahmedabad" }
  },
  {
    product_id: 5,
    name: "T-Shirt",
    category: "Clothing",
    price: 800,
    stock: 6,
    supplier: { supplier_name: "FashionHub", contact: "9090909090", address: "Surat" }
  },
  {
    product_id: 6,
    name: "Jeans",
    category: "Clothing",
    price: 1500,
    stock: 12,
    supplier: { supplier_name: "FashionHub", contact: "9090909090", address: "Surat" }
  },
  {
    product_id: 7,
    name: "Notebook",
    category: "Books",
    price: 100,
    stock: 4,
    supplier: { supplier_name: "BookStore", contact: "9001122334", address: "Ahmedabad" }
  }
])
```

## (c) Retrieve Products with Stock < 10

```
> db.products.find({ stock: { $lt: 10 } })
< {
    _id: ObjectId('699ed05dbebb98459ec5d30a'),
    product_id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 50000,
    stock: 5,
    supplier: {
      supplier_name: 'TechSupplier',
      contact: '9876543210',
      address: 'Mumbai'
    }
  }
  {
    _id: ObjectId('699ed05dbebb98459ec5d30c'),
    product_id: 3,
    name: 'Headphones',
    category: 'Electronics',
    price: 3000,
    stock: 8,
    supplier: {
      supplier_name: 'SoundHub',
      contact: '9988776655',
      address: 'Pune'
    }
  }
```

```
  {
    _id: ObjectId('699ed05dbebb98459ec5d30e'),
    product_id: 5,
    name: 'T-Shirt',
    category: 'Clothing',
    price: 800,
    stock: 6,
    supplier: {
      supplier_name: 'FashionHub',
      contact: '9090909090',
      address: 'Surat'
    }
  }
  {
    _id: ObjectId('699ed05dbebb98459ec5d310'),
    product_id: 7,
    name: 'Notebook',
    category: 'Books',
    price: 100,
    stock: 4,
    supplier: {
      supplier_name: 'BookStore',
      contact: '9001122334',
      address: 'Ahmedabad'
    }
  }
```

## (d) Increase Price by 10% for a Category

```
> db.products.updateMany(
    { category: "Electronics" },
    { $mul: { price: 1.1 } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 3,
    modifiedCount: 3,
    upsertedCount: 0
  }
```

```
> db.products.find({ category: "Electronics" })
< {
    _id: ObjectId('699ed05dbebb98459ec5d30a'),
    product_id: 1,
    name: 'Laptop',
    category: 'Electronics',
    price: 55000.00000000001,
    stock: 5,
    supplier: {
      supplier_name: 'TechSupplier',
      contact: '9876543210',
      address: 'Mumbai'
    }
  }
  {
    _id: ObjectId('699ed05dbebb98459ec5d30b'),
    product_id: 2,
    name: 'Mobile',
    category: 'Electronics',
    price: 22000,
    stock: 15,
    supplier: {
      supplier_name: 'GadgetWorld',
      contact: '9123456789',
      address: 'Delhi'
    }
  }
```

**Practical 12:**
**(a) Create a 'library' collection with fields: book_id, title, author (array of authors), ISBN, published_year, genres**
**(array), and availability status.**
**(b) Insert sample book documents with multiple authors and genres.**
**(c) Query books by a specific author using array query operators.**
**(d) Find all books published between 2015 and 2023 and belonging to specific genres using $and, $gte, and $lte**
**operators.**
**(e) Create a compound index on title and published_year fields.**

## (a) Create library Collection Structure

MongoDB creates collection automatically on insert.

### (b) Insert Sample Books

```
> db.library.insertMany([
    {
      book_id: 1,
      title: "MongoDB Basics",
      author: ["John Smith", "Alex Brown"],
      ISBN: "978-1111111111",
      published_year: 2020,
      genres: ["Technology", "Database"],
      available: true
    },
    {
      book_id: 2,
      title: "NodeJS Guide",
      author: ["Emily Davis"],
      ISBN: "978-2222222222",
      published_year: 2018,
      genres: ["Programming", "Technology"],
      available: true
    },
    {
      book_id: 3,
      title: "Web Development",
      author: ["John Smith", "Sara Lee"],
      ISBN: "978-3333333333",
      published_year: 2016,
      genres: ["Programming", "Web"],
      available: false
    },
```

```
    {
      book_id: 4,
      title: "Cloud Computing",
      author: ["Michael Scott"],
      ISBN: "978-4444444444",
      published_year: 2022,
      genres: ["Technology", "Cloud"],
      available: true
    },
    {
      book_id: 5,
      title: "Modern JavaScript",
      author: ["Emily Davis", "John Smith"],
      ISBN: "978-5555555555",
      published_year: 2023,
      genres: ["Programming", "Technology"],
      available: true
    }
  ])
```

### (c) Query Books by Specific Author

```
> db.library.find({ author: "John Smith" })
< {
    _id: ObjectId('699ed17fbebb98459ec5d311'),
    book_id: 1,
    title: 'MongoDB Basics',
    author: [
      'John Smith',
      'Alex Brown'
    ],
    ISBN: '978-1111111111',
    published_year: 2020,
    genres: [
      'Technology',
      'Database'
    ],
    available: true
  }
```

```
{
  _id: ObjectId('699ed17fbebb98459ec5d313'),
  book_id: 3,
  title: 'Web Development',
  author: [
    'John Smith',
    'Sara Lee'
  ],
  ISBN: '978-3333333333',
  published_year: 2016,
  genres: [
    'Programming',
    'Web'
  ],
  available: false
}
{
  _id: ObjectId('699ed17fbebb98459ec5d315'),
  book_id: 5,
  title: 'Modern JavaScript',
  author: [
    'Emily Davis',
    'John Smith'
  ],
  ISBN: '978-5555555555',
  published_year: 2023,
  genres: [
    'Programming',
    'Technology'
  ],
  available: true
}
```

### (d) Books Between 2015–2023 & Specific Genres

```
> db.library.find({
    $and: [
      { published_year: { $gte: 2015, $lte: 2023 } },
      { genres: "Programming" }
    ]
  })
< {
    _id: ObjectId('699ed17fbebb98459ec5d312'),
    book_id: 2,
    title: 'NodeJS Guide',
    author: [
      'Emily Davis'
    ],
    ISBN: '978-2222222222',
    published_year: 2018,
    genres: [
      'Programming',
      'Technology'
    ],
    available: true
  }
```

```
{
  _id: ObjectId('699ed17fbebb98459ec5d313'),
  book_id: 3,
  title: 'Web Development',
  author: [
    'John Smith',
    'Sara Lee'
  ],
  ISBN: '978-3333333333',
  published_year: 2016,
  genres: [
    'Programming',
    'Web'
  ],
  available: false
}
{
  _id: ObjectId('699ed17fbebb98459ec5d315'),
  book_id: 5,
  title: 'Modern JavaScript',
  author: [
    'Emily Davis',
    'John Smith'
  ],
  ISBN: '978-5555555555',
  published_year: 2023,
  genres: [
    'Programming',
    'Technology'
  ],
  available: true
}
```

### (e) Create Compound Index

```
> db.library.createIndex({ title: 1, published_year: 1 })
< title_1_published_year_1
```

**Practical 13:**

**(a) Using the 'orders' collection, write aggregation queries to group orders by customer_id and calculate total order**
**count per customer.**

**(b) Use the $match stage to filter orders above a certain price threshold.**

**(c) Use $project stage to display only specific fields (customer_id, product, total_price).**

**(d) Combine multiple aggregation stages to find customers who made more than 3 orders.**

### (a) Group Orders by customer_id & Count Orders

```
> db.orders.aggregate([
    {
      $group: {
        _id: "$customer_id",
        totalOrders: { $sum: 1 }
      }
    }
  ])
< {
    _id: 102,
    totalOrders: 1
  }
  {
    _id: 103,
    totalOrders: 1
  }
  {
    _id: 101,
    totalOrders: 2
  }
```

### (b) Filter Orders Above Certain Price (Using $match)

```
> db.orders.aggregate([
    {
      $match: { price: { $gt: 10000 } }
    }
  ])
< {
    _id: ObjectId('699ecd411206048fd4f0a8b8'),
    order_id: 1,
    customer_id: 101,
    product: 'Laptop',
    quantity: 1,
    price: 50000,
    order_date: 2025-02-10T00:00:00.000Z
  }
  {
    _id: ObjectId('699ecd411206048fd4f0a8b9'),
    order_id: 2,
    customer_id: 102,
    product: 'Mobile',
    quantity: 2,
    price: 20000,
    order_date: 2025-02-15T00:00:00.000Z
  }
  {
    _id: ObjectId('699ecd411206048fd4f0a8ba'),
    order_id: 3,
    customer_id: 101,
    product: 'Tablet',
    quantity: 1,
    price: 15000,
    order_date: 2025-03-01T00:00:00.000Z
  }
```

### (c) Use $project to Display Selected Fields

```
> db.orders.aggregate([
    {
      $project: {
        customer_id: 1,
        product: 1,
        total_price: { $multiply: ["$quantity", "$price"] },
        _id: 0
      }
    }
  ])
< {
    customer_id: 101,
    product: 'Laptop',
    total_price: 50000
  }
  {
    customer_id: 102,
    product: 'Mobile',
    total_price: 40000
  }
  {
    customer_id: 101,
    product: 'Tablet',
    total_price: 15000
  }
  {
    customer_id: 103,
    product: 'Headphones',
    total_price: 9000
  }
```

### (d) Customers Who Made More Than 3 Orders

```
> db.orders.aggregate([
    {
      $group: {
        _id: "$customer_id",
        totalOrders: { $sum: 1 }
      }
    },
    {
      $match: {
        totalOrders: { $gt: 3 }
      }
    }
  ])
```

(no output if no customer has more than 3 orders)

**Practical 14:**

**(a) Create a 'blog' collection with fields: post_id, title, content, author_id, tags (array), comments (array of embedded documents with comment_id, user_id, comment_text, timestamp), created_at, updated_at.**

**(b) Insert blog posts with nested comment arrays.**

**(c) Add a new comment to an existing blog post using array update operators ($push).**

**(d) Remove a specific comment from a blog post using $pull.**

**(e) Query all blog posts that contain a specific tag using array operators.**

### (a) Blog Collection Structure

MongoDB creates collection automatically on insert.

### (b) Insert Blog Posts with Nested Comments

```
> db.blog.insertMany([
    {
      post_id: 1,
      title: "MongoDB Basics",
      content: "Introduction to MongoDB",
      author_id: 101,
      tags: ["database", "mongodb"],
      comments: [
        {
          comment_id: 1,
          user_id: 201,
          comment_text: "Great post!",
          timestamp: new Date()
        }
      ],
      created_at: new Date(),
      updated_at: new Date()
    },
```

```
    {
      post_id: 2,
      title: "NodeJS Guide",
      content: "Learn NodeJS step by step",
      author_id: 102,
      tags: ["nodejs", "javascript"],
      comments: [
        {
          comment_id: 1,
          user_id: 202,
          comment_text: "Very helpful",
          timestamp: new Date()
        }
      ],
      created_at: new Date(),
      updated_at: new Date()
    }
  ])
```

### (c) Add New Comment using $push

```
> db.blog.updateOne(
    { post_id: 1 },
    {
      $push: {
        comments: {
          comment_id: 2,
          user_id: 203,
          comment_text: "Thanks for sharing",
          timestamp: new Date()
        }
      }
    }
  )
```

### (d) Remove Specific Comment using $pull

```
> db.blog.updateOne(
    { post_id: 1 },
    {
      $pull: {
        comments: { comment_id: 1 }
      }
    }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

### (e) Query Blog Posts Containing Specific Tag

```
> db.blog.find({ tags: "mongodb" })
< {
    _id: ObjectId('699ed36ed8ecdbd6d3160ed5'),
    post_id: 1,
    title: 'MongoDB Basics',
    content: 'Introduction to MongoDB',
    author_id: 101,
    tags: [
      'database',
      'mongodb'
    ],
    comments: [
      {
        comment_id: 2,
        user_id: 203,
        comment_text: 'Thanks for sharing',
        timestamp: 2026-02-25T10:48:45.728Z
      }
    ],
    created_at: 2026-02-25T10:48:14.879Z,
    updated_at: 2026-02-25T10:48:14.879Z
  }
```

**Practical 15:**

**(a) Implement text search functionality by creating a text index on the 'title' and 'content' fields of the blog**
**collection.**

**(b) Perform text search queries to find blog posts containing specific keywords.**

**(c) Use projection to display search results with relevance scores.**

**(d) Combine text search with other query conditions (e.g., author_id and date range).**

### (a) Create Text Index on title and content

```
> db.blog.createIndex(
    { title: "text", content: "text" }
  )
< title_text_content_text
```

### (b) Perform Text Search Queries

```
> db.blog.find(
    { $text: { $search: "MongoDB" } }
  )
< {
    _id: ObjectId('699ed36ed8ecdbd6d3160ed5'),
    post_id: 1,
    title: 'MongoDB Basics',
    content: 'Introduction to MongoDB',
    author_id: 101,
    tags: [
      'database',
      'mongodb'
    ],
    comments: [
      {
        comment_id: 2,
        user_id: 203,
        comment_text: 'Thanks for sharing',
        timestamp: 2026-02-25T10:48:45.728Z
      }
    ],
    created_at: 2026-02-25T10:48:14.879Z,
    updated_at: 2026-02-25T10:48:14.879Z
  }
```

### (c) Display Results with Relevance Score

```
> db.blog.find(
    { $text: { $search: "MongoDB" } },
    {
      score: { $meta: "textScore" },
      title: 1,
      content: 1
    }
  ).sort({ score: { $meta: "textScore" } })
< {
    _id: ObjectId('699ed36ed8ecdbd6d3160ed5'),
    title: 'MongoDB Basics',
    content: 'Introduction to MongoDB',
    score: 1.5
  }
```

### (d) Combine Text Search with Other Conditions

```
> db.blog.find(
    {
      $text: { $search: "MongoDB" },
      author_id: 101,
      created_at: { $gte: ISODate("2024-01-01") }
    },
    {
      title: 1,
      score: { $meta: "textScore" }
    }
  )
< {
    _id: ObjectId('699ed36ed8ecdbd6d3160ed5'),
    title: 'MongoDB Basics',
    score: 1.5
  }
```